

计算机视觉课程课后作业二

建智 2201 谢天赐 20221082044

问题一：



图 1

经过网上查找，收集到各种环境条件下的直道、弯道图片各五张。为了避免其他信息对于图片识别的影响，通过前期预处理进行了简单裁剪以及无关水印的去除结果如图 1 所示。

问题二：

针对问题要求，完善了 myCh02.py 文件中的“多图像读取显示 T”函数，部分代码如图 2 所示。

```
def ch02_多图像读取显示T():  
    ccmsg = "请 选择/输入 将要打开的图像文件名..."  
    fnPict = tkFileDialog.askopenfilenames(title=ccmsg, filetypes=[('Image files', '*.jpg *.png *.bmp'), ('All files', '*.*')])  
  
    numPict = len(fnPict)  
    myImageAR = []  
    for i in range(numPict):  
        fn = fnPict[i]  
        img0 = Image.open(fn)  
        img = ImageTk.PhotoImage(img0)  
        myImageAR.append(img)  
  
    rootwin = tk.Toplevel()  
    rootwin.title("浮动窗体(多图像读取显示T)")  
    # 根据图片数量调整窗体大小  
    rootwin.geometry(f"{numPict * 200}x{600+100+100}")  
  
    # 每列4张图片  
    numCols = 4  
    numRows = (numPict + numCols - 1) // numCols # 计算需要多少行  
  
    # 创建水平的PanedWindow  
    myWinHor = PanedWindow(rootwin, orient=HORIZONTAL)  
    myWinHor.pack(fill=BOTH, expand=1)  
  
    for i in range(numRows):  
        # 创建垂直的PanedWindow
```

图 2

这段代码通过循环，对每个选择的文件路径 fnPict[i]，使用 Pillow 库

Image.open(fn)打开图片并将每张图片调整大小到 150x150 像素，以适应 GUI 显示。同时，设置每列显示 4 张图片。计算需要多少行来显示所有图片。使用了 PanedWindow 创建一个水平的容器，用于容纳垂直的 PanedWindow。对于每一行，创建了一个垂直的 PanedWindow 并添加到水平容器中。在每个垂直的 PanedWindow 中，循环添加图片和标题。如果图片数量不足一行的 4 张，使用空白标签填充。对于每张图片，创建一个 Label 组件来显示图片。设置 Label 的 image 属性，以保持对图片的引用。为每张图片创建一个标题 Label，显示文件名（不包括路径和扩展名）。如果图片数量不足以填满最后一行，使用空白 Label 来保持网格对齐。

运行结果如图 3 所示，在完成要求的标准上保证了美观性。

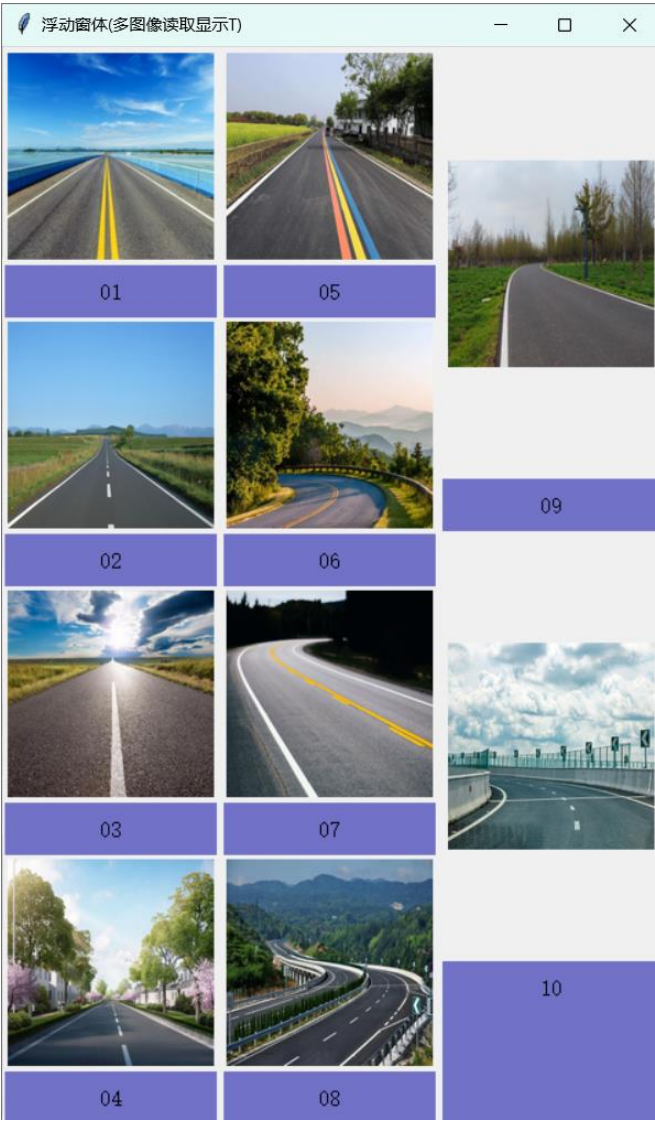


图 3

问题三：

针对问题三要求，在 myCh02.py 文件中增加函数如图 4 所示。

```

# 打开图像并显示(多个)，使用matplotlib显示灰度图和直方图
def ch02_多图像读取显示M():
    # 选择图像文件
    image_paths = filedialog.askopenfilenames(title="Select images", filetypes=[("Image files", "*.jpg *.png *.bmp"), ("All files", "*.*")])

    # 确保最多只有4张图
    if len(image_paths) > 4:
        image_paths = image_paths[:4]

    # 设置matplotlib的图形大小
    plt.figure(figsize=(12, 8))

    for i, image_path in enumerate(image_paths):
        # 读取图像
        img = cv2.imread(image_path)
        # 转换为灰度图
        gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # 绘制原始图像
        plt.subplot(3, 4, i+1)
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.title("Original")
        plt.axis('off')

        # 绘制灰度图
        plt.subplot(3, 4, i+5)
        plt.imshow(gray_img, cmap='gray')
        plt.title("Gray")

```

图 4

这段代码用于从用户选择的多个图像文件中读取图像，显示原始图像、转换为灰度图后的图像以及灰度图的直方图。使用了 matplotlib 和 cv2（即 OpenCV 库）来处理 and 显示图像。下面对一些重要的语句进行解释：

- `if len(image_paths) > 4:` 检查用户选择的图像数量是否超过 4 张。
- `image_paths = image_paths[:4]:` 超过 4 张只保留前 4 张图像的路径。
- `gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY):` 将读取的图像转换为灰度图。
- `plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)):` 显示原始图像。由于 OpenCV 读取的图像是 BGR 格式，而`matplotlib`期望的是 RGB 格式，所以需要转换。
- `plt.imshow(gray_img, cmap='gray'):` 显示灰度图。
- `plt.subplot(3, 4, i+1+8):` 设置下一个子图的位置，用于显示直方图。
- `plt.hist(gray_img.ravel(), 256, [0, 256]):` 计算并绘制灰度图的直方图。
- `plt.tight_layout():` 自动调整子图参数，使之填充整个图形区域。

可见，本代码对用户选择图像的数量做了意外处理，具有一定的鲁棒性，同时考虑到了 OpenCV 读取图像与 matplotlib 格式的差别，严谨而有逻辑。

运行结果如图 5 所示：

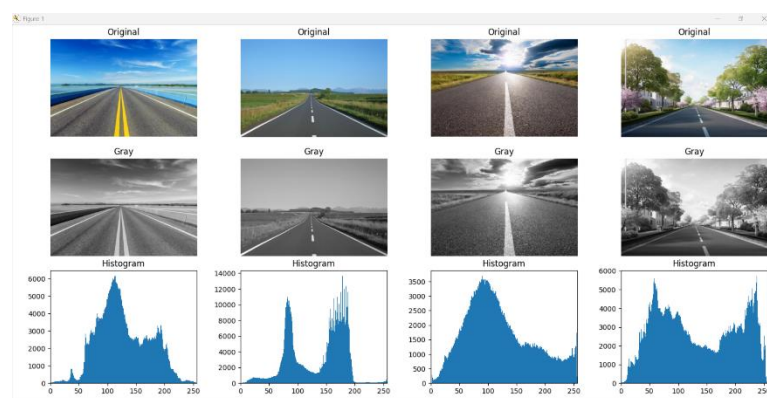


图 5

问题四——研究摘要：

本研究（作业）旨在通过收集和处理道路中直道和弯道图片，以及实现图像的读取、显示和灰度转换，来加深对计算机视觉技术在图像处理中应用的理解。通过对不同环境下的直道与弯道的处理

问题一：通过网上搜索收集了直道和弯道的图片，并进行了裁剪和去水印的预处理。**问题二：**使用 Python 的 Pillow 库调整图片大小，并利用 Tkinter 的 PanedWindow 组件在 GUI 中以网格形式显示图片。**问题三：**结合使用 matplotlib 和 OpenCV 库，实现了图像的读取、灰度转换和直方图的绘制。

研究通过将道路分为直道和弯道，可以更精确地提取道路特征，为后续的图像分析和处理提供基础。通过对不同类型道路的图像分析，可以训练机器学习模型以识别和区分直道和弯道，这对于自动驾驶系统尤为重要。为自动驾驶和道路监控系统提供丰富的训练数据，直道和弯道的图像及其灰度图和直方图可以作为特征用于机器学习模型的训练。研究直道和弯道的视觉感知差异，有助于理解人类驾驶员在不同道路条件下的视觉处理机制，进而设计更符合人类视觉特性的辅助驾驶系统。为驾驶员提供实时的道路信息，如弯道的曲率和直道的延伸方向，有助于做出更安全的驾驶决策。

十张图的直方图对比结果如下所示：

